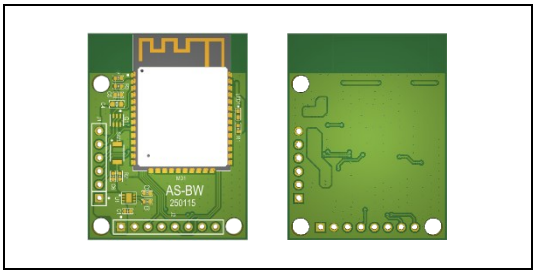
AS_BW

Inzinious Radar Linked Bluetooth / WiFi Module
Easy connection, easy use
GPIO, UART, Indicate LED

User Manual



1 Features

- Multi-protocol IoT gateway bridging AnySensing radar modules to server / cloud infrastructure
- Bluetooth / WiFi one-board solution
- Includes antenna and DSP
- Wide input voltage available: 3.3~7.0V
- Compact size and easy to use
- GPIO and UART interface
- Low power consumption: 700mW
- Provides a variety of sensing modes
  - Easily connects with any AS radar via 2.54mm pins
  - WiFi setting function available via Bluetooth
  - Data bypass available when used with IoT devices

2 Specification

Content	Definition
Model name	AS-BW
Hardware size	32.0 x 40.0 x 4.25 mm (No connector)
Weight	-
Wireless	2.4 GHz WiFi, 802.11b/g/n + BLE 4.2
Supply voltage	3.3 ~ 7 V DC (regulated)
Supply current	≈ 700 mW typical; up to ≈ 1.0 W during WiFi TX bursts
Security	BLE passkey (6-digit), WPA2, TLS 1.2, mTLS
Operating / Storage temperature	-20 ~ 60 ℃ / -40 ~ 85 ℃
Interface	1 GPIO, 1 UART(TTL level)
Function	Motion, Distance, Vital-sign(BR, HR, HRV), Indicate LED
MCU	ESP32 (Dual-core Xtensa LX6, 240 MHz)
PSRAM	2 MB
Flash	8 MB internal (dual 3 MB OTA slots, 1 MB SPIFFS file system)
UART baud	9600 / 19200 / 38400 / 57600 / 115200 / 230400 / 256000 / 460800 / 921600 bps (configurable, default 460800)

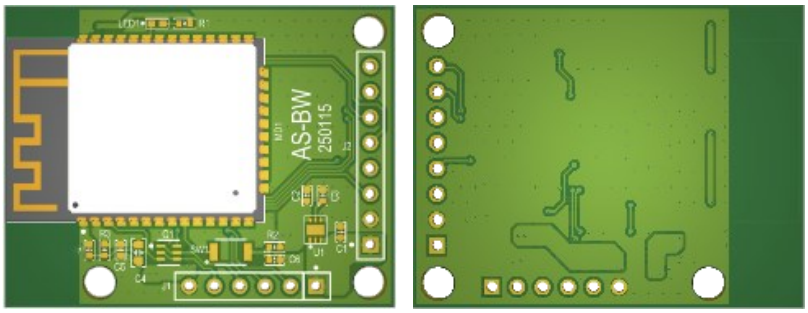
3 Pin Description

Pin	Signal	Direction	Function
1	VCC_5V	Power in	Main supply input. See Specification for the accepted voltage range
2	GND	Power	Ground
3	GPIO_EX0	In	Optional radar detect line (driven HIGH by the radar on detection). Off by default
4	GPIO_EX1	I/O	Expansion GPIO. Not used by the shipping firmware
5	UART1_TX	Out	Radar UART transmit. Wire to the radar module's RX
6	UART1_RX	In	Radar UART receive. Wire to the radar module's TX
7	UART2_TX	Out	Expansion UART transmit. Not used by the shipping firmware
8	UART2_RX	In	Expansion UART receive. Not used by the shipping firmware

- All external connections are made through J2, a single-row 8-pin 2.54 mm header (TSW-108-08-G-S). Direction is given from the AS-BW's point of view: Out = driven by AS-BW, In = driven by the connected device.
- Signal pins are 3.3 V TTL levels.
- Pins 5 / 6 are the radar link. To an AS-614, AS-613 or AS-622: J2-5 → radar RX, J2-6 → radar TX, plus a common ground on J2-2.
- Pins 4 / 7 / 8 are routed to the header for future expansion.
- Status LED is Active-Low, PWM 5 kHz / 8-bit, at a fixed intensity of 5/255. The led command only switches it off and on.
- The radar NRST line (internally KILL_GPIO) is driven open-drain and is not used during OTA. Radar OTA enters the bootloader over UART; the line stays released (high-Z) and is pulsed LOW only by the radarnrst command.
- USB-Serial console operates at 115200 8N1.

4 Dimensions

- Compact 32.0 × 40.0 × 4.25 mm two-layer PCB. A 2.54 mm pin header on the bottom edge provides all electrical connections. The on-board PCB trace antenna is on the top edge. Keep a 10 mm RF clearance from metal enclosures or large ground planes for best WiFi / BLE range.



(left) Top view: ESP32 module, antenna, status LED.
(right) Bottom view: 2.54 mm pin header. Pin 1 marker on the silkscreen.

5 System Configuration

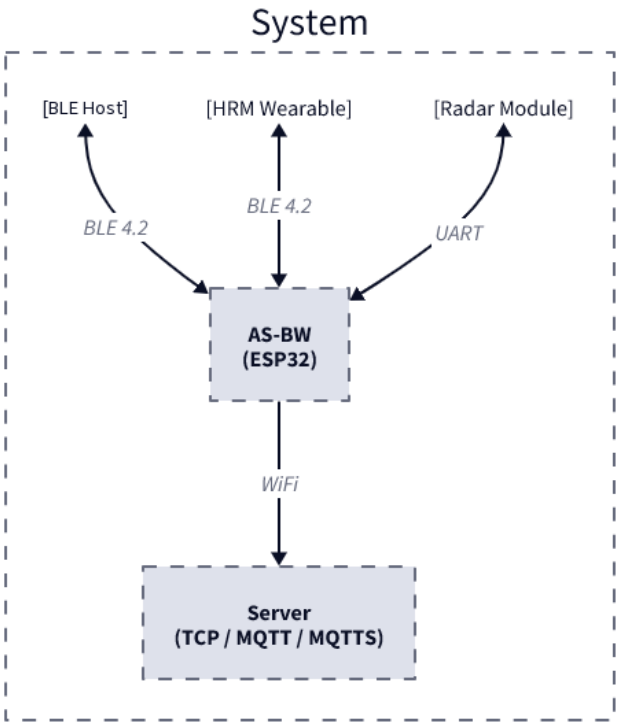


Figure: AS-BW system topology.

- One AS-BW pairs with one radar module via UART.
- Radar payload is broadcast in real time on the active server transport.
- BLE (Peripheral role) provides configuration and live data.
- WiFi operates in STA mode only (no AP / SoftAP).
- The AS-BW remembers the pairing, so the passkey is asked once per host rather than on every connection.

6 Functions

Transport / Feature	Description
TCP Socket	Persistent TCP over AsyncTCP. Bidirectional command channel.
MQTT	MQTT 3.1.1 over plain TCP. User / password auth, configurable QoS 0/1/2, Last Will and Testament supported. Bidirectional command channel: a payload prefixed __bw on the subscribe topic runs as a gateway command and the reply is published to <subscribe topic>/bw. Other payloads forward to the radar.
MQTTS	MQTT 3.1.1 over TLS 1.2. CA certificate verification plus optional mTLS client cert. SHA-1 fingerprint pinning available. Commands sent to the subscribe topic prefixed __bw are answered on <subscribe topic>/bw.
BLE Configuration	Settings are read / written over the custom BLE service. 6-digit passkey pairing.
BLE Live Data	Radar payload streamed over the Notify characteristic to the paired host.
BLE File Transfer	Chunked binary upload over two dedicated characteristics on the primary service, with a per-packet CRC16 and JSON acknowledgements.
Radar OTA	Radar firmware is uploaded over WiFi to the gateway's HTTP update server, then proxied to the radar UART bootloader. Radar data forwarding pauses for the duration of the update.
ESP32 OTA	The BLE OTA characteristics start the on-device HTTP update server and report progress; the firmware image itself is uploaded over WiFi. Written to inactive slot, image validated, boot-slot switched on success. A boot guard then puts the new image on probation until it has run for 60 seconds. During that window, four boots that each end in a fault (crash, watchdog or power brown-out) make the gateway switch back to the previous image by itself. Ordinary restarts and power cycles are not counted, and once the image has proven itself nothing is counted at all. It is always on and needs no setup, so a unit reporting an older version than the one you pushed has rolled back, and that image should be treated as bad.
Diagnostics Black Box	Device log mirrored to on-board flash (a live file plus one rotated backup, 128 KB each), surviving a power cycle. After an abnormal reset (panic, watchdog, or brownout) the gateway sends a single ;C,<reason>,<core0>/<core1>,<uptime>,<minHeap>! frame on the next server connection; normal reboots stay silent. Read the log back with seriallog.
Fleet (ESP-NOW)	Optional peer-to-peer link between AS-BW units sharing the same WiFi SSID (and therefore the same 2.4 GHz channel). Radar lines are broadcast to up to 8 listed peers; frames received from siblings are relayed to a subscribed host over the Fleet Data BLE characteristic. Disabled by default; enable with fleet 1.

- WPA2 only. WPA3-only networks are not supported.
- WiFi 5 GHz networks are not supported. STA mode operates on 2.4 GHz only.
- MQTTS requires a CA certificate when CA use is enabled. The device falls back to the Mozilla CA bundle if no custom CA is loaded.
- Sending commands over MQTT requires the MAC prefix to be on (macprefix 1, the default). The device publishes the identity from which a host derives this unit's command topic only while the prefix is enabled.
- mqttqos, mqttkeepalive and mqttlwt take effect on the next MQTT (re)connect, not immediately. Issue reset, or switch protocol away and back, to apply them at once.

- certchunk is TCP only. pushbw and pushradar run on every channel: BLE (plain text only, not the JSON envelope), Serial, TCP and MQTT. Their progress replies reach the live uplink and the log path (Serial + BLE) together, so the channel that issued the command always sees the outcome; pushradar's intermediate flash-progress percentages also reach every channel, but may be dropped.
- mTLS requires both client certificate and private key uploaded to the device, via uploadcert (BLE) or certchunk (TCP).
- Maximum decoded certificate size is 8192 bytes per file. Single-PEM only: full chains exceeding the limit must be trimmed to the leaf or root needed.
- The device stores up to 8 BLE bonds. When a 9th host pairs, the oldest bond is dropped automatically and that host must pair again; send bleclear to clear them all.
- Radar OTA pauses UART forwarding for the duration of the update.
- ESP32 OTA reboots automatically on successful update. If the updated firmware then fails to stay running (four crashes before it manages one minute of uptime), the device reverts to the previous image by itself. Check the reported version after an update.
- Preventive auto-reboot is enabled by default. The device reboots itself after 1 day of uptime to clear long-run WiFi / Bluetooth coexistence and memory fragmentation. Each unit is offset by a MAC-derived 0 ~ 2 h delay so a fleet does not reboot in lockstep, and the reboot is deferred while an ESP32 OTA or radar update is in progress. Send autoreboot <0-7> to change the cadence in days; 0 disables it.

7 Bluetooth Information

- The AS-BW advertises as AS-BW_<5hex>, where <5hex> is the last 5 hex digits of the unit's MAC address. When the device is powered it can be found with any BLE scanner.
- Pairing requires a 6-digit BLE passkey. The factory value is given in the shipping documentation and can be changed with the blepass command.
- The AS-BW remembers the pairing, so the passkey is asked once per host rather than on every connection. To pair from a different host, send bleclear, or freset to also restore the factory settings.
- Note: if the device settings are changed the device may reboot. Bluetooth then needs to be reconnected as well.

Bluetooth settings for integration

- The AS-BW is compliant with the BLE standard GATT. Every characteristic below belongs to the single vendor-specific primary service, including the two file-transfer characteristics, which borrow the Nordic UART UUIDs. There is no separate Nordic UART Service in the GATT database. A standard Device Information Service (180A) is also present.

Characteristic	UUID	Properties	Function
Service	0000FFF0-0000-1000-8000-00805F9B34FB	–	Vendor-specific primary service
Manage	AAAAAAAA-0000-1000-8000-00805F9B34FB	Write	WiFi / server / protocol settings. Writing saves the

			settings and reboots the device.
Connect	BBBBBBBB-0000-1000-8000-00805F9B34FB	Write	WiFi connection control
Data	CCCCCCCC-0000-1000-8000-00805F9B34FB	Read / Notify	Real-time sensor data stream
Status	DDDDDDDD-0000-1000-8000-00805F9B34FB	Read / Notify	Device status (JSON)
Command	EEEEEEEE-0000-1000-8000-00805F9B34FB	Write / Notify	Command execution and reply
SSIDList	FFFF0000-0000-1000-8000-00805F9B34FB	Read / Notify	WiFi scan results
OTA Status	FFFF1111-0000-1000-8000-00805F9B34FB	Read / Notify	Firmware update progress
OTA Command	FFFF2222-0000-1000-8000-00805F9B34FB	Read / Write / Notify	Firmware update trigger (start / stop / status / radar)
Fleet Data	FFFF3333-0000-1000-8000-00805F9B34FB	Read / Notify	Relayed sibling-unit frames (ESP-NOW fleet mode).
File RX	6E400002-B5A3-F393-E0A9-E50E24DCCA9E	Write	Chunked file upload, client to AS-BW
File TX	6E400003-B5A3-F393-E0A9-E50E24DCCA9E	Notify	Transfer acknowledgement / progress, AS-BW to client

- File transfer uses two packet types on File RX, metadata then data, with a CRC16 over each data payload; File TX returns a JSON acknowledgement, progress and completion for every packet. Firmware images are not carried here; they are uploaded over WiFi to the HTTP update server. Note also that no characteristic supports Write Without Response: every write must be sent as a write-with-response, or it is discarded silently.

8 Setup

- Power on the AS-BW. The status LED indicates the running state.
- From a BLE host, scan for the device named AS-BW_<5hex>, where <5hex> is the last 5 hex digits of the unit's MAC address.
- Pair using the 6-digit BLE passkey. The factory passkey is given in the shipping documentation; change it with the blepass command.
- Set the WiFi SSID / password and the target server address over BLE.
- The device connects automatically on the next boot.

Reboot and factory reset

Command	Action
reset	Reboot only. All saved settings are preserved.
freset	Restore factory defaults (HRM settings, autoreboot, health counters and the saved serial log are kept), wipe the BLE bond store, delete the stored MQTT / MQTTS certificates, then reboot.

- Both commands are accepted over BLE, the USB-Serial console, or the server connection.

- Issue reset after changing a major setting such as the active protocol or the server endpoint. State left over from the previous configuration can otherwise cause unexpected behaviour.

9 Data Output Format

- All output is ASCII. Data frames end with the '!' character; command replies end with LF (₩n).

Radar frame

- Radar UART output is forwarded to the active transport as [MAC_PREFIX]<radar_payload>!

Item	Byte	Content
MAC_PREFIX	5	Last 5 characters of the device MAC address, uppercase. Optional; enabled by default and shared by TCP, MQTT and MQTTS.
<radar_payload>	var	Pass-through ASCII from the radar UART. The content depends on the radar firmware mode.
!	1	Character indicating the end of data

- Example: with MAC suffix A1B2C and radar payload 1,0,000,00,00,000 the device outputs A1B2C1,0,000,00,00,000!
- Trailing newline characters are stripped from the radar payload before TCP / MQTT / MQTTS transmission; the BLE Data characteristic carries the full line, its terminator kept as the radar sent it. The MAC prefix is controlled by one setting shared by TCP, MQTT and MQTTS.

Frame start and end characters

Channel	Start	End
Radar payload	MAC prefix, or the first payload byte	!
Crash breadcrumb	;C,	!
Device identity	<MAC5>/id;	!
Command reply (Serial / BLE)	=	LF (₩n)
Command reply (TCP)	[BW]	LF (₩n)
Command reply (MQTT / MQTTS)	[BW]	LF (₩n), on <subscribe topic>/bw
Heartbeat	00;	!

10 Protocol

Communication method

- The AS-BW accepts commands over the USB-Serial console, the BLE Command characteristic, the TCP connection and the MQTT / MQTTS subscribe topic. Data is sent and received as ASCII character codes.
- All commands must be in lowercase.
- Serial, BLE and TCP commands must be terminated with LF (₩n). A CR just before it is ignored, so CRLF works; a CR alone does not end a command. An MQTT message is one command and needs no terminator.
- Settings are stored in non-volatile memory and restored automatically after a power cycle.
- Allow at least 200 ms between consecutive commands to avoid response collisions.

Channel	Command format	Reply
USB-Serial console	Plain text, 115200 8N1	= ... LF (₩n)
BLE Command characteristic	Plain text ending in LF, or JSON envelope, e.g. ver or {"command":"ver"}	= ... LF (₩n)
TCP	__bw <command>	[BW] ... LF (₩n)
MQTT / MQTTS	__bw <command> on the subscribe topic	[BW] ... on <subscribe topic>/bw

UART settings (radar link)

Item	Setting value
Baud rate	460,800 bps (default). Configurable: 9600 / 19200 / 38400 / 57600 / 115200 / 230400 / 256000 / 460800 / 921600 (baud)
Data	8 bit
Parity bit	None
Stop bits	1 bit
Flow control	None
Idle flush timeout	0 ms: flush on newline only (configurable 0 ~ 5000 ms)

Server transport

Item	TCP	MQTT	MQTTS
Default endpoint	192.168.0.46 : 9000	115.68.193.126 : 1883	115.68.193.126 : 8883
Protocol stack	AsyncTCP	MQTT 3.1.1	MQTT 3.1.1 over TLS 1.2
Keep-alive	30 s TCP keepalive probe; closed after 3 missed	60 s	60 s
Authentication	–	User / password	CA certificate and optional client certificate / key
Health probe	00;!	00;! and Last Will	00;! and Last Will
Client identifier	–	Full 12-hex MAC, lowercase. mqttclientid overrides.	Full 12-hex MAC, lowercase. mqttclientid overrides.
QoS	–	1 by default (mqttqos 0 / 1 / 2). Used for publish, subscribe and the Last Will.	1 by default (mqttqos 0 / 1 / 2). Used for publish, subscribe and the Last Will.
Session	–	Clean session (since v4.7). The broker keeps nothing across a drop: a command sent while the unit is offline is dropped, not replayed. Resend it once the Last Will topic reads online. The device re-subscribes on every connect.	Clean session (since v4.7). The broker keeps nothing across a drop: a command sent while the unit is offline is dropped, not replayed. Resend it once the Last Will topic reads online. The device re-subscribes on every connect.
Identity on connect	<MAC5>/id;<model>;mac=XX:XX:XX:XX:XX:XX!	<MAC5>/id;<model>;mac=<12-hex>! published retained to the publish topic.	<MAC5>/id;<model>;mac=<12-hex>! published retained to the publish topic.
Birth / Last Will	–	Retained online on connect, retained offline	Retained online on connect, retained offline

		on unexpected loss.	on unexpected loss.
Reconnect back-off	5 / 10 / 20 / 40 / 60 s cap, ±25 % jitter	5 / 10 / 20 / 40 / 60 s cap, ±25 % jitter	5 / 10 / 20 / 40 / 60 s cap, ±25 % jitter

MQTT topics

- Four topics are used. Three are settable; the command-reply topic is derived from the subscribe topic and cannot be set.

Role	Setting	Default	Direction and content
Publish (data)	mqttpubtopic	defaultSTopic	AS-BW to server. Radar frames, the 00;! heartbeat, the retained identity and the crash breadcrumb.
Subscribe (command)	mqttsubtopic	(empty): the full 12-hex MAC, lowercase	Server to AS-BW. A payload starting __bw runs as a gateway command; any other payload is forwarded to the radar UART unchanged.
Command reply	(derived)	<subscribe topic>/bw	AS-BW to server. [BW] ... replies, LF-terminated. Kept off the data topic so radar consumers never see command traffic.
Birth / Last Will	mqttlwt	(empty): <12-hex MAC>/status	AS-BW to broker. Retained online on connect; retained offline delivered by the broker on unexpected loss.

- mqttpubtopic, mqttsubtopic and mqttlwt accept a literal {MAC} token, replaced at connect / publish with this device's full MAC rendered per macfmt, e.g. 33/AS/{MAC} becomes 33/AS/4C-7C-5F-0B-54-05 with the default upper-dash format.
- macfmt takes <upper/lower>-<dash/colon/dot/underscore/none>. The separators /, # and + are intentionally not offered: / would split the MAC across topic levels, and # and + are MQTT wildcards that are illegal in a published topic.
- A topic containing no {MAC} is used literally, byte for byte, so an existing fixed-topic deployment is unaffected by macfmt.
- The default subscribe and Last Will topics (empty setting) use the bare lowercase 12-hex MAC, which differs from a {MAC}-rendered topic (uppercase and dashed by default). MQTT topics are case-sensitive, so use one convention per fleet.
- On every connect, and only while macprefix is 1, the gateway publishes <MAC5>/id;<model>;mac=<12-hex MAC>! retained to the publish topic. A host that subscribes after the device is already connected still receives it and derives this unit's command topic from the mac= trailer. With macprefix 0 the publish degrades to id;<model> with no MAC, and per-device command targeting over MQTT stops working.

TCP socket session

- On connect the device sends one identity frame, <MAC5>/id;<model>;mac=XX:XX:XX:XX:XX:XX! It holds the last 5 hex characters of the

MAC in uppercase, the advertised device name, and the full MAC uppercase and colon-separated. It ends with ! and carries no newline. A crash breadcrumb ;C,...! follows once if the previous boot ended abnormally.

- The device then streams radar frames, and beats a bare 00;! every 60 s ± 15 s. A 00;! sent by the server is echoed back verbatim as a liveness probe.
- Command replies are [BW] <key>=<value> terminated by a bare LF. Earlier firmware used CRLF; the CR was dropped so every channel now terminates replies the same way. A multi-line reply such as status is written line-atomically: a line that does not fit the send buffer whole is dropped rather than truncated, so re-poll on a missing reply.
- Reconnection uses a 5 to 60 s exponential back-off with ±25 % jitter.

11 Command Reference

Command	Channel	Default Value	Explanation
Flow control Commands			
help	All	—	Print the list of supported commands.
ver	All	—	Print the firmware version: 14-digit build stamp (used to match OTA updates), build variant, and display version.
status	All	—	Return current device status (WiFi link, server connection, radar, heap, lifecycle, fleet, OTA) as JSON.
config	All	—	Dump every saved configuration parameter and its current value.
reset	All	—	Reboot only. NVS preserved.
freset	All	—	Factory reset. Restore NVS settings to factory defaults (HRM settings, autoreboot, health counters and the saved serial log are kept), wipe the BLE bond store, delete the stored MQTT/MQTTS certificates, and reboot.
autoreboot <0-7>	All	1	Set the preventive auto-reboot cadence. The device reboots after this many days of uptime to clear accumulated WiFi / memory issues: 0 disables. The reboot is offset per unit (0 ~ 2 h) so a fleet does not reboot in lockstep, and is never taken during an OTA or radar update. Persists across reboot. Range: 0 ~ 7 days.
System / Diagnostics			
heap	All	—	Show free heap, max contiguous block, minimum-ever, and fragmentation.
led	All	1	Show LED enable state (1 = enabled, 0 = disabled).
led <0/1>	All	—	Disable (0) or enable (1) the status LED. Setting persists across reboot.
wink	All	—	Blink the status LED hard and wink the radar for ~5 s to physically identify this unit.
seriallog	All	—	Replay the saved serial log to the USB serial console. Reply is the byte count. The log is timestamped, stored on flash, auto-rotates (up to about 256 KB retained), and is kept across a power cycle.
seriallog size	All	—	Show the saved log size and rotation generation: size=<bytes>,gen=<n>.
seriallog get <off> <len>	All	len ≤ 512	Fetch one base64-encoded window of the saved serial log, for paging the log to a host over BLE / TCP. In the reply, n=0 marks the end of the log; a changed gen value means the log rotated and paging must be restarted.
seriallog clear	All	—	Erase the saved serial log.
WiFi commands			
wifissid <SSID>	All	≤32 chars	Save the target WiFi SSID to non-volatile memory. A factory-reset unit carries bench-provisioning credentials. Set the deployment's own network before use.
wifipass <pass>	All	≤63 chars	Save the WiFi password to non-volatile memory. Replaced together with the SSID after a factory reset.
wificonnect <SSID> <pass>	All	—	Connect immediately to the specified network without saving credentials.
wifidisconnect	All	—	Drop the current WiFi connection.
wifiscan	All	—	Trigger a WiFi scan. Results delivered via SSIDList characteristic or TCP.
Server / Protocol commands			
protocol <proto>	All	normalsocket	Select the active uplink transport.
tcpserver <host>	All	192.168.0.46	Set the TCP socket server address.
tcpport <port>	All	9000	Set the TCP socket server port.

macprefix <0/1>	All	1	Disable (0) or enable (1) the frame MAC prefix: the last 5 hex characters of the MAC, uppercase. One switch shared by TCP, MQTT and MQTTS. Must stay enabled for per-device command targeting over MQTT.
mqttserver <host>	All	115.68.193.126	Set the MQTT / MQTTS broker address.
mqttport <port>	All	1883 (MQTT) / 8883 (MQTTS)	Set the MQTT / MQTTS broker port.
mqttpubtopic <topic>	All	defaultSTopic	Set the publish topic. May contain a {MAC} token, replaced at each publish with this device's full MAC rendered per macfmt (e.g. 33/AS/{MAC} to 33/AS/4C-7C-5F-0B-54-05). A topic with no {MAC} is used literally. This topic also carries the retained identity publish and the 00:! heartbeat.
mqttsubtopic <topic>	All	(empty)	Set the subscribe topic. Radar commands arrive here; a payload prefixed __bw is handled as a gateway command and its reply is published to <topic>/bw. Left empty, the device subscribes to its own full 12-hex MAC.
mqttclientid <id>	All	(MAC-derived)	Set the MQTT client identifier. When left empty the device's full 12-hex MAC is used.
mqttuser <user>	All	—	Set the MQTT username.
mqttpass <pass>	All	—	Set the MQTT password.
mqttqos <0-2>	All	1	Set the QoS used for publish, subscribe and the Last Will. Applied on the next MQTT (re)connect.
mqttkeepalive <sec>	All	60	Set the broker keepalive (PINGREQ) interval in seconds; 0 uses the library default. Applied on the next MQTT (re)connect. Range: 0 ~ 3600.
mqttlwt <topic>	All	(empty)	Set the Last Will topic. May contain a {MAC} token rendered per macfmt. mqttlwt -1 clears it and restores the default <full 12-hex MAC>/status; an empty argument only reads the current value. Applied on the next MQTT (re)connect.
mqttsca <0/1>	All	0	Disable / enable CA certificate validation.
mqttsmtls <0/1>	All	0	Disable / enable mTLS client certificate.
mqttsfp <sha1>	All	—	Set MQTTS broker SHA-1 fingerprint.
macfmt <value>	All	upper-dash	Set how the {MAC} token in MQTT topics is rendered: <upper/lower>-<dash/colon/dot/underscore/none>. Default upper-dash renders as 4C-7C-5F-0B-54-05.
idfullmac <0/1>	All	0	Use the full 12-hex MAC in the BLE / Bluetooth device name (e.g. AS-BW_4C7C5F0B5405) instead of the 5-hex suffix (AS-BW_B5405). Takes effect after reboot.
profile <tag>	All	(empty)	Show or set a free-text profile / customer tag identifying this unit. Reported by config and status only, and does not affect any topic or payload format. profile -1 clears it; an empty argument only reads it. Persists across reboot.
Radar commands			
radar	All	—	Show radar updater status (busy flag).
radarcmd <cmd>	All	—	Forward a command string to the radar over UART.
radarsim [ttl=<sec>] <p1> ... <p8>	All	—	Simulate the radar: loop 1 to 8 fake frames into the radar-to-server path at 1 Hz, one per second, repeating, so three payloads give a three-second cycle. They reach the server exactly like real radar frames, so a server can be tested with no radar attached. While the loop runs, frames from a real radar are muted so that only the simulated sequence is delivered. The loop stops by itself after 300 seconds so a unit is never left muted by accident. Add ttl=<sec> to change that, or ttl=0 to run until stopped. radarsim 0 stops the loop and restores the real radar; radarsim on its own reports the current state. Payloads are separated by spaces, so a payload cannot contain one. Does not touch the radar UART. Range: 1 ~ 8 payloads, each 1 ~ 200 chars. Renamed from testdata, which injected a single frame.
radarnrst	All	—	Hardware-reset the radar via the NRST pin. Works even when the radar is unresponsive on UART. Refused during radar OTA. Returns queued right away; the reset or noeffect result is reported separately once the reset completes.
radarbaud	All	—	Show the current radar UART application baud rate.
radarbaud <rate>	All	460800	Set the radar UART application baud rate. Accepts 9600 / 19200 / 38400 / 57600 / 115200 / 230400 / 256000 / 460800 / 921600. An invalid rate is refused immediately; a valid one is applied a moment later and confirmed.
radarbootbaud	All	—	Show the current radar bootloader baud rate.
radarbootbaud <rate>	All	460800	Set the radar bootloader baud rate. Used during OTA.
radartestcrc	All	—	Run the CRC algorithm self-test. Returns PASS or FAIL.
radarstream <0/1>	All	0	Disable (0) or enable (1) echo of raw radar UART bytes for debugging. USB-Serial always receives a verbatim byte echo. Issued over TCP/MQTT, each assembled raw line is additionally mirrored to the __bw management channel as [BW] radarstream=<payload>, so a unit with no USB attached can be debugged remotely; not mirrored to BLE. The reply reports the active sink (sink=off|usb|usb+tcp|usb+mqtt). Volatile: resets on reboot.
uartflushms	All	—	Show the current UART idle-flush timeout (ms).

uartflushms <ms>	All	0	Set the UART idle-flush timeout. 0 = flush on ₩n only. Range: 0 ~ 5000.
OTA commands			
ota	All	—	Show OTA HTTP server status and URL.
ota <0/1>	All	0	Start / stop the OTA HTTP server. (TCP channel: triggers OTA check.)
pushradar [613|614|622] <url>	All	—	Download radar firmware from <url> and flash it to the radar over UART. The optional 613, 614 or 622 selects the radar model; without it the current model setting applies, which is AS-614 after a reboot. AS-613 and AS-622 accept encrypted .sbin images only. The intermediate flash-progress percentages reach every channel but may be dropped; the final done / error reply reaches every channel, including the BLE link that issued the command.
pushbw <url>	All	—	Download AS-BW firmware from <url> and self-update the gateway. Reboots on success. Also accepted over BLE (as plain text, not the JSON envelope), which is how a site with no WiFi is updated: send wificonnect to put the unit on a laptop or phone hotspot, then pushbw with that host's URL. Progress is reported back on the same BLE link, and several units can pull at once.
Fleet commands			
fleet	All	—	Show fleet mode status (enabled, active, peer count, tx / rx counters).
fleet <0/1>	All	0	Disable (0) or enable (1) ESP-NOW fleet mode. Applied immediately, no reboot.
fleetpeers <csv>	All	(empty)	Set the fleet peer MAC list. Up to 8 comma-separated MACs; fleetpeers -1 clears the list (an empty argument only reads it).
Security commands			
blepass <6digits>	All	123456	Set the BLE pairing passkey. Range: 1 ~ 6 decimal digits (0 ~ 999999). Takes effect immediately.
bleclear	All	—	Erase all BLE bonds; the device will re-pair on next connection.
uploadcert	BLE	—	Upload a PEM certificate to SPIFFS via BLE (JSON, base64). See section 8.4.
certchunk <t> <s> <tot> <b64>	TCP	—	Upload a PEM certificate in base64 chunks over the TCP management channel. TCP equivalent of uploadcert. Not available over MQTT.

Warning

The device operates wirelessly and can generate radio waves that can prevent the pacemaker or implantable medical device from functioning properly. Do not use the device within 20 cm of a pacemaker or other implantable medical device. When using a pacemaker or other implantable medical device, do not use the device without first consulting your doctor or the manufacturer of the pacemaker or other implantable medical device.

Disclaimer

The information herein is believed to be correct as of the date issued. ANYSENSING and/or INZINIOUS (“INZINIOUS”) will not be responsible for damages of any nature resulting from the use or reliance upon the information contained herein. INZINIOUS makes no warranties, expressed or implied, of merchantability or fitness for a particular purpose or course of performance or usage of trade. Therefore, it is the user’s responsibility to thoroughly test the product in their particular application to determine its performance, efficacy, and safety. Users should obtain the latest relevant information before placing orders.
Unless INZINIOUS has explicitly designated an individual INZINIOUS product as meeting the requirements of a particular industry standard, INZINIOUS is not responsible for any failure to meet such industry standard requirements.

Unless explicitly stated herein, INZINIOUS has not performed any regulatory conformity tests. It is the user's responsibility to ensure that necessary regulatory conditions are met and approvals have been obtained when using the product. Regardless of whether the product has passed any conformity test, this document does not constitute any regulatory approval of the user's product or application using INZINIOUS's product.
Nothing contained herein is to be considered as permission or a recommendation to infringe any patent or any other intellectual property right. No license, express or implied, to any intellectual property right is granted by INZINIOUS herein.
INZINIOUS reserves the right to correct, change, amend, enhance, modify, and improve this document and/or INZINIOUS products at any time without notice.
This document supersedes and replaces all information supplied prior to the publication hereof.